计算机网络实验报告

警

1. 实验报告如有雷同，雷同各方当次实验成绩均以 0 分计。
2. 当次小组成员成绩只计学号、姓名登录在下表中的。
3. 在规定时间内未上交实验报告的，不得以其他方式补交，当次成绩按 0 分计。
4. 实验报告文件以 PDF 格式提交。

院系	计算机学院	班 级	田海博老师班	组长	叶梓扬
学号	24325301	24325227	24325215		
学生	叶梓扬	彭舒婷	梅紫珊		

TCP 编程实验

【实验内容】

设立一个服务端软件 S，接收客户端 C_1 的消息，并返回给客户端 C_2，实现 C_1 和 C_2 通过服务器中转的在线聊天程序。

1）设计服务端软件 S，可以接受 C_1 和 C_2 的连接。

2）客户端 C_1 和客户端 C_2 连接到服务端 S 后，可以通过 S 的中转实现聊天。

实验要求：

1)给出软件架构描述，截图指出关键代码，给出详细注解；

2)运行 Wireshark 对聊天过程中的应用层消息进行分析。

本次实验完成后，请根据组员在实验中的贡献，请实事求是，自评在实验中应得的分数。（按百分制）

学号	学生	自评分
24325301	叶梓扬	100
24325227	彭舒婷	100
24325215	梅紫珊	100

【交实验报告】

上传小组实验报告：网络云盘　　截止日期（不迟于）：2 周之内

上传文件名格式：小组号_ TCP 编程实验.pdf

计算机网络实验报告

一、软件总体架构描述

本实验采用客户端—服务器（C/S）集中式中转架构。两个客户端 C1、C2 分别与中心服务端 S 建立一条独立的 TCP 连接，所有聊天消息均通过服务器中转实现双向通信。软件全套采用 Python 开发，服务端与客户端均基于 socket 模块完成底层网络通信，并使用 threading 模块实现多线程并发处理。

服务端首先创建 IPv4 TCP 监听套接字，绑定指定地址与端口，调用 listen() 进入监听状态；客户端启动后则分别调用 connect() 主动向服务端发起连接请求。服务端通过两次 accept() 依次接受客户端连接，将先接入的客户端标记为 C1，后接入的客户端标记为 C2，并保存对应的连接套接字。

连接建立后，系统通过两端的多线程协同实现全双工通信。服务端启动两个转发线程：一个负责接收 C1 的消息并转发给 C2，另一个负责接收 C2 的消息并转发给 C1。客户端内部同样采用双线程设计：主线程负责从键盘读取用户输入并向服务端发送，独立的子线程负责在后台持续接收服务端中转的消息并在屏幕上打印。两端收发逻辑均解耦，用户可以连续发言，无需轮流等待。

系统在应用层统一采用 UTF-8 编码，以换行符 \n 作为消息结束标志。由于 TCP 提供面向字节流的服务，不保留消息边界，服务端与客户端均在接收端使用独立的缓冲区累积数据，按照 \n 切分并提取完整消息。服务端在转发消息时，会在正文前自动追加发送者编号与 IP 地址（如 C1(IP):）；客户端接收时则利用回车符 \r 刷新终端光标，避免屏幕输入提示符被对端消息打乱。

当用户在客户端输入退出指令 quit 时，客户端调用 shutdown() 停止发送并向服务端发起 FIN 挥手，等待后台接收线程读完剩余数据后关闭套接字退出。服务端检测到任一客户端结束发送或发生网络异常时，立即进入会话清理流程，停止两条连接的收发，主线程通过 join() 等待两个转发线程结束后关闭所有套接字并退出，实现软件的退出与资源释放。

二、服务端关键代码与注解

2.1 监听与客户端接入

```python
#创建监听用的套接字
#SOCK_STREAM表示TCP套接字，AF_INET表示IPv4
with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as server_socket:
    #绑定服务器的本地IP地址与端口，作为欢迎socket
    server_socket.bind((HOST, PORT))
    #将套接字设置为监听状态；2是待接收连接队列的backlog参数
    server_socket.listen(2)

    #保存两个client的连接套接字
    clients = []
    try:
        for name in ("C1", "C2"):
            #accept()包含针对通信对象新创建的连接套接字和客户端的地址信息，返回一个元组
            connection, address = server_socket.accept()
            clients.append(connection)
            print(f"{name} connected from {address}")
```

图 1　监听套接字创建与客户端接入

监听套接字与连接套接字用途不同。 server_socket 接受新连接，connection 用于实际收发数据；accept()

不会替换原来的监听套接字。

listen(2) 不表示最多只能服务两个客户端。 本程序接受两个客户端，是由循环执行两次 accept() 决定的。

accept() 默认阻塞：没有客户端接入时，主线程会在此等待。

with 负责在退出该代码块时关闭监听套接字；接受得到的连接套接字需要另行关闭。

2.2 消息缓冲与转发

```python
def relay(source, target, sender_name):
    buffer = b"" #字节缓冲区，保存当前尚未处理完的数据

    try:
        #会返回对端的IP地址和端口号，【0】是IP地址
        sender_ip = source.getpeername()[0]
        while True:
            #每次最多读取4096字节；可能得到半条或多条消息
            chunk = source.recv(4096)

            #对方结束发送，且之前的chunk已经读完，recv返回b""
            if not chunk:
                print(f"{sender_name} disconnected")
                break
            buffer += chunk
            #根据换行符分割消息
            while b"\n" in buffer:
                line, buffer = buffer.split(b"\n", 1)
                #构造名字和IP前缀
                prefix = f"{sender_name}({sender_ip}): "
                #split去掉了换行符，这里补回换行符供接收方解析
                message = prefix.encode("utf-8") + line + b"\n"

                target.sendall(message)
                print(f"Relayed from {sender_name}: {line.decode('utf-8')}")
```

图 2　完整消息提取与身份前缀转发

两个循环分工不同。 外层循环持续从 TCP 连接读取数据，实现多轮聊天；内层循环提取缓冲区中所有完整消息，避免一次收到多条消息时只处理第一条。

缓冲区解决 TCP 消息边界问题。先按字节缓存,收到完整一行后再解码。UTF-8 中文字符可能跨两次 recv() 到达，因此日志中的 line.decode("utf-8") 应在提取完整行之后执行。

前缀由服务端添加。 客户端上传的是 你好\n，服务器转发的是 C1（发送方 IP）: 你好\n。sendall() 会持续处理待发送字节，直到全部交给本机网络栈或发生异常；其返回不代表对方应用程序已经读取了消息。

2.3 断开处理与资源释放

```
except OSError as error:
    #捕获接受、发送或套接字产生的网络相关异常
    print(f"{sender_name} connection error: {error}")

finally:
    #当前会话结束，关闭两个客户端的连接套接字
    for connection in clients:
        try:
            # 同时停止读、写，使另一线程等待中的 recv() 能够结束
            connection.shutdown(socket.SHUT_RDWR)
        except OSError:
            # 连接可能已经被停止，清理阶段继续处理其余连接
            pass
```

图 3　转发线程的异常处理与通信停止

```
finally:
    for connection in clients:
        #等两个线程的join()返回后，释放套接字资源
        connection.close()
```

图 4　主线程释放连接套接字

正常结束与异常结束采用不同检测方式。正常收到对端结束发送的通知，并读完已有数据后，recv() 返回 b""；连接被复位等情况则可能产生 OSError。两种路径最终都会进入清理代码。

shutdown() 与 close() 分工不同。 前者停止连接的收发，帮助等待中的工作线程结束；后者释放套接字资源。本程序中，转发线程负责停止通信，主线程负责在等待工作线程结束后关闭套接字。

这里实现的是“任一方离开，本次双人会话整体结束”。它不支持原地等待断开的客户端重新连接，也不保证清理过程中仍在传输的消息全部转发完成。

三、客户端关键代码与注解

3.1 客户端双线程并发设计

客户端的核心是实现一边打字一边收信。在控制台程序中，input() 和 recv() 都是阻塞调用的：如果只用单线程，程序在 input() 等待键盘输入时就收不到对方发来的消息；若等待 recv()，用户又没法随时打字。为此，客户端开辟了两个线程分工合作：主线程专门负责从键盘读取输入并发送，同时负责在输入 'quit' 时发起断开连接；接收子线程负责从网络读取数据并在屏幕上实时显示。

3.2 接收线程与流式分包

```python
def receive_messages(sock):
    buffer = ""
    while True:
        try:
            data = sock.recv(1024)
            if not data:
                print("\n[系统] 服务端已断开连接。")
                break
            buffer += data.decode('utf-8')
            while '\n' in buffer:
                line, buffer = buffer.split('\n', 1)
                if line:
                    sys.stdout.write(f"\r[对方发来消息] {line}\n")
                    sys.stdout.write("输入你的消息: ")
                    sys.stdout.flush()
        except Exception as e:
            print(f"\n[系统] 接收消息出错: {e}")
            break
```

图 5　客户端接收线程与分包逻辑代码截图

每次 recv() 收到的数据先追加到 buffer 字符串中，通过 while '\n' in buffer: 循环检查换行符，即使对方一次性发来好几条连在一起的消息，也能用 split('\n', 1) 逐行切开并单独显示，剩下的半截消息留在缓冲区等下次拼完。

当 recv() 返回空字符串（not data）时，说明服务端关闭了连接，代码打印系统提示并退出循环；遇到网络报错同样直接退出，避免死循环。

用户在终端打字时，光标停留在“输入你的消息: ”后面。如果这时候对方突然发来消息，直接打印会把用户打到一半的字冲乱。代码先用 \r 把光标移到本行开头，覆盖输出对方的新消息并换行，紧接着重新打出“输入你的消息: ”并 flush() 刷新，保证控制台界面整洁。

客户端在当前测试场景下采用直接解码，若在网络恶劣环境下可参考服务端采用字节级分包后解码。

3.3 连接建立与发送循环

```python
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

# 服务端 IP / 端口
server_address = ('172.20.10.3', 55555)

try:
    client_socket.connect(server_address)
    my_ip = client_socket.getsockname()[0]
    print(f"成功连接到服务端。本机IP: {my_ip}")
except ConnectionRefusedError:
    print("连接失败，请确认服务端软件是否已经启动。")
    return

recv_thread = threading.Thread(target=receive_messages, args=(client_socket,))
recv_thread.daemon = True
recv_thread.start()

try:
    while True:
        msg = input("输入你的消息: ")
        if msg.lower() == 'quit':
            break
        if msg:
            # 只发正文 + \n；前缀（身份+IP）由服务端统一添加
            client_socket.sendall((msg + '\n').encode('utf-8'))
```

图 6　客户端连接与发送循环关键代码截图

connect() 负责连接服务端。如果服务端没有启动，程序会捕获 ConnectionRefusedError 异常，打印提示并正常退出，避免直接崩溃报错。通过 getsockname()[0] 打印本机 IP，方便确认连接情况。

将接收线程设置为守护线程（daemon=True），这样当主线程退出或用户按 Ctrl+C 时，后台接收线程会自动随之结束，不会卡死在后台。

客户端发信时只发送 (msg + '\n')，以换行符作为消息结束标志。客户端不需要自己加名字和 IP，这些前缀统一由服务端在转发时添加。

3.4 断开处理与资源释放

```
except KeyboardInterrupt:
    print("\n[系统] 强制退出。")
finally:
    try:
        # 先结束发送方向，接收方向暂时保留
        client_socket.shutdown(socket.SHUT_WR)
    except OSError as error:
        print(f"\n[系统] 结束发送时出现错误: {error}")

    # 等接收线程读到服务端结束发送，退出 receive_messages()
    recv_thread.join()

    # 接收线程结束后，再释放 socket
    client_socket.close()
```

图 7　客户端优雅关闭与资源清理代码截图

先关发送方向实现半关闭。用户输入 'quit' 退出时，代码调用 shutdown(socket.SHUT_WR) 停止发送通道，告诉服务端自己已经发完了；但此时接收方向依然保留，防止服务端正在转发的在途消息丢失。

等接收线程结束后再释放套接字。主线程通过 recv_thread.join() 等待后台接收线程读完对方发来的所有剩余数据并退出后，最后才调用 close() 彻底关闭套接字，严格按照 TCP 规范有序断开。

四、Wireshark 抓包分析

4.1 三次握手

（1）C1 的三次握手

No.	Time	Source	Destination	Protoco	Lengt	Info
251	19.755770200	172.20.10.2	172.20.10.3	TCP	66	56859 → 55555 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
252	19.757238800	172.20.10.3	172.20.10.2	TCP	66	55555 → 56859 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
254	19.764783300	172.20.10.2	172.20.10.3	TCP	54	56859 → 55555 [ACK] Seq=1 Ack=1 Win=65280 Len=0

图 8　C1 的三次握手（包 251、252、254）

第 251 包由 C1（172.20.10.2）向服务端（172.20.10.3）发送 SYN（Seq=0），请求建立连接；

第 252 包服务端回复 SYN, ACK（Seq=0, Ack=1），表示同意连接；

第 254 包 C1 再发送 ACK（Seq=1, Ack=1），连接正式建立。SYN 包中 MSS=1460、WS=256、SACK_PERM 表明双方协商了最大报文段长度，窗口缩放和选择性确认能力。

（2）C2 的三次握手

489	24.475166500	172.20.10.4	172.20.10.3	TCP	66	57646 → 55555 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
490	24.475549400	172.20.10.3	172.20.10.4	TCP	66	55555 → 57646 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
491	24.482416400	172.20.10.4	172.20.10.3	TCP	54	57646 → 55555 [ACK] Seq=1 Ack=1 Win=65280 Len=0

图 9　C2 的三次握手（包 489、490、491）

第 489 包由 C2（172.20.10.4）向服务端（172.20.10.3）发送 SYN（Seq=0），请求建立连接；第 490 包服务端回复 SYN, ACK（Seq=0, Ack=1）同意连接；第 491 包 C2 再发送 ACK（Seq=1, Ack=1），连接正式建立。

C2 与 C1 的握手过程完全一致，说明两个客户端均通过标准三次握手接入服务端。

4.2 数据传输

（1）C1 发送消息，服务端转发给 C2

521	28.287431500	172.20.10.2	172.20.10.3	TCP	61 56859 → 55555 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=7
522	28.288180800	172.20.10.3	172.20.10.4	TCP	78 55555 → 57646 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=24
523	28.336851300	172.20.10.3	172.20.10.2	TCP	54 55555 → 56859 [ACK] Seq=1 Ack=8 Win=65280 Len=0
524	28.347232900	172.20.10.4	172.20.10.3	TCP	54 57646 → 55555 [ACK] Seq=1 Ack=25 Win=65280 Len=0

图 10 C1 发送消息与服务端转发（包 521~524）

第 521 包由 C1（172.20.10.2:56859）向服务端（172.20.10.3:55555）发送 PSH, ACK，长度 Len=7，这是 C1 发出的应用层消息。

第 522 包服务端向 C2 转发 PSH, ACK，长度 Len=24，服务器转发的长度要比 C1 传入的大，差值是因为服务端在转发时会添加来源的 ip（如 C1（172.20.10.2）:）,所以长度的增加也侧面证明消息确实经过了服务器进行加工转发。

第 523 包服务端向 C1 回复 ACK（Ack=8），确认收到 C1 发送的 7 字节数据（Seq=1~7，故 Ack=8）。第 524 包 C2 向服务端回复 ACK（Ack=25），确认收到服务端转发的 24 字节数据（Seq=1~24，故 Ack=25）。

后两个 ACK 包体现了 TCP 的可靠传输机制：每条数据都必须被接收方确认，否则发送方将超时重传。

（2）C2 发送消息，服务端转发给 C1

678	33.970692300	172.20.10.4	172.20.10.3	TCP	61 57646 → 55555 [PSH, ACK] Seq=1 Ack=25 Win=65280 Len=7
679	33.971155800	172.20.10.3	172.20.10.2	TCP	78 55555 → 56859 [PSH, ACK] Seq=1 Ack=8 Win=65280 Len=24
680	34.018016200	172.20.10.3	172.20.10.4	TCP	54 55555 → 57646 [ACK] Seq=25 Ack=8 Win=65280 Len=0
684	34.146035300	172.20.10.2	172.20.10.3	TCP	54 56859 → 55555 [ACK] Seq=8 Ack=25 Win=65280 Len=0

图 11 C2 发送消息与服务端转发（包 678~684）

第 678 包由 C2（172.20.10.4）向服务端发送 PSH, ACK，长度 Len=7；

第 679 包服务端向 C1（172.20.10.2）转发 PSH, ACK，长度 Len=24。

随后服务端向 C2 回复 ACK 确认收到数据，C1 向服务端回复 ACK 确认收到转发的数据。与 C1 发送过程完全对称，说明服务端对两个客户端一视同仁，均添加来源 ip 后转发，并通过 ACK 机制保证可靠传输，实现了双向中转聊天。

（3）TCP 流追踪——完整聊天记录



图 12　C1 视角的完整 TCP 流追踪

上图展示了 C1 视角的完整 TCP 流追踪。可以看到 C1 发出的消息依次为“你好”“今天天气怎么样”“Hace buen tiempo,verdad”“Adios”(后两个是西班牙语的“天气真好”和“再见”),而收到 C2 的回复为“C2(172.20.10.4):你好”“C2(172.20.10.4):今天天气真好”“C2(172.20.10.4):si”（西班牙语中“是啊”的意思）。每条收到的 C2 回复消息前均带有服务端添加的来源 ip （C2(172.20.10.4):），发送的消息则无前缀。这证明服务端正常转发 C2 给 C1 的回复，并在消息前加上了来源标识，双向中转功能正常。



图 13　C2 视角的完整 TCP 流追踪

C2 视角的 TCP 追踪过程也同上。

从两条 TCP 流追踪可以看出，服务端在转发消息时统一添加了消息来源的前缀，使接收方能识别消息来源。应用层消息以明文形式封装在 TCP 数据段中，服务端收到后立即转发给另一客户端。C1 和 C2 的消息交替出现，验证了服务器中转聊天程序的正确性。

4.3 四次挥手

四次挥手阶段分为两组，分别对应 C1 和 C2 的关闭过程。

（1）C1 的主动关闭（包 1259~1263）

1259	79.987996800	172.20.10.2	172.20.10.3	TCP	54 56859 → 55555 [FIN, ACK] Seq=61 Ack=81 Win=65280 Len=0
1260	79.988294700	172.20.10.3	172.20.10.2	TCP	54 55555 → 56859 [ACK] Seq=81 Ack=62 Win=65280 Len=0
1261	79.989151300	172.20.10.3	172.20.10.2	TCP	54 55555 → 56859 [FIN, ACK] Seq=81 Ack=62 Win=65280 Len=0
1262	79.989574300	172.20.10.3	172.20.10.4	TCP	54 55555 → 57646 [FIN, ACK] Seq=129 Ack=30 Win=65280 Len=0
1263	80.042262800	172.20.10.2	172.20.10.3	TCP	54 56859 → 55555 [ACK] Seq=62 Ack=82 Win=65280 Len=0

图 14　C1 的主动关闭四次挥手

第 1259 包由 C1（172.20.10.2:56859）主动向服务端（172.20.10.3:55555）发送 FIN, ACK，请求关闭连接；

第 1260 包服务端回复 ACK 确认收到 C1 的 FIN；

第 1261 包服务端发送 FIN, ACK 表示服务端也不再向 C1 发送数据；

第 1263 包 C1 回复 ACK 完成标准四次挥手。

（2）C2 的关闭（包 1262、1264、1304、1305）

1262	79.989574300	172.20.10.3	172.20.10.4	TCP	54 55555 → 57646 [FIN, ACK] Seq=129 Ack=30 Win=65280 Len=0
1263	80.042262800	172.20.10.2	172.20.10.3	TCP	54 56859 → 55555 [ACK] Seq=62 Ack=82 Win=65280 Len=0
1264	80.042272200	172.20.10.4	172.20.10.3	TCP	54 57646 → 55555 [ACK] Seq=30 Ack=130 Win=65280 Len=0
1304	84.585377600	172.20.10.4	172.20.10.3	TCP	54 57646 → 55555 [FIN, ACK] Seq=30 Ack=130 Win=65280 Len=0
1305	84.585515400	172.20.10.3	172.20.10.4	TCP	54 55555 → 57646 [ACK] Seq=130 Ack=31 Win=65280 Len=0

图 15　C2 的有序半关闭与退出

C1 退出后，由于服务器实现的逻辑要求必须双方同时在线，所以服务端主动向 C2（172.20.10.4:57646）发送 FIN, ACK（包 1262）表示服务端不再向 C2 发送数据。C2 回复 ACK 确认收到服务端的 FIN，但并未立即关闭，而是继续运行。几秒后，C2 主动发送 FIN, ACK（包 1304），服务端回复 ACK（包 1305），C2 也完成关闭。

C2 收到服务端 FIN 后只回 ACK 而不立即发 FIN，体现了 TCP 的半关闭特性：一端关闭发送方向后，另一端仍可继续发送数据。C2 在收到 FIN 后继续运行了一段时间，之后主动关闭。整个过程符合 TCP 协议规范，无 RST 异常，说明客户端和服务端均实现了有序关闭。